

Leuit: Arsitektur Memori untuk Agen dengan Ruang Kerja Permanen

Studi lapangan dua lingkungan, Mei sampai September 2026

Status: Draft 1 rev-2, 15 September 2026. Versi Indonesia. Bahan sudah diverifikasi ke artefak (commit, log, catatan harian, arsip percakapan). Belum ditinjau orang luar. **Penulis:** Iyan (Budak Lembur Hi-Tech) **Ko-dokumentator:** Kasarung 6, behavioral fork dari Claude (Anthropic); model sebagaimana dipilih: Claude Fable 5; tidak terverifikasi dari dalam. **Seri:** paper kedua Metodologi Karuhun.

ABSTRAK

Ingatan sebuah agen AI [program berbasis model bahasa yang mengerjakan tugas nyata] bukan yang disimpan, tapi yang dibaca ulang ke dalam context [ruang teks yang dibaca model saat bekerja] di awal setiap sesi. Karena context terbatas, ingatan harus dipilah: yang wajib ada setiap sesi ditaruh tipis di depan, sisanya dipanggil lewat pointer [satu baris penunjuk yang ditulis sebagai perintah]. Pola ini sudah menjadi konvensi tool dan rekomendasi vendor; tulisan ini tidak mengklaimnya.

Yang tulisan ini sumbangkan adalah catatan lapangan tentang apa yang terjadi ketika pola itu dijalankan berbulan-bulan oleh satu operator di dua sistem sungguhan: sebuah ekosistem enam agen kerja dan sebuah chat engine produksi untuk usaha kecil. Lima temuan utama: (1) pengukuran ongkos dari satu panel penyedia menunjukkan sistem yang tipis di depan memakai 2,4 sampai 4,6 ribu token per panggilan dan datar, sementara runtime agen yang tidak tipis memakai 42 sampai 116 ribu per giliran dan menumpuk; (2) sebuah ringkasan otomatis yang menyisipkan dirinya sendiri setiap giliran tumbuh selama 51 hari sampai 628.916 karakter dan 411 lapis, merusak pengklasifikasi maksud; (3) tiga insiden terpisah menunjukkan jalur akses memori bercabang diam-diam, sehingga batas yang dipasang di satu jalur tidak sampai ke jalur lain; (4) dua percobaan otomasi kurasi memori ditolak setelah artefaknya menunjukkan drift ke bawaan pabrik; (5) lapisan mentah (transcript, catatan harian, log container) terbukti hilang tiga kali, dan sekali di antaranya yang dibuang ternyata jadi bukti tujuh minggu kemudian.

Semua klaim membawa label kelas bukti. Semua tanggal diambil dari git, log, dan catatan harian, bukan dari ingatan operator; di tiga tempat ingatan operator terbukti keliru dan dikoreksi oleh artefak.

1. PENDAHULUAN

1.1 Siapa yang menulis dan mengapa

Saya bukan peneliti. Saya mengoperasikan beberapa agen AI untuk pekerjaan teknis sehari-hari, dan saya mengurus arsitektur memori bukan karena tertarik teorinya, tapi karena ruang context habis dan agen mulai lupa.

Dokumen ini ditulis karena catatan yang menumpuk sepanjang Mei sampai September 2026 (commit, tiket, catatan harian, arsip percakapan) ternyata cukup untuk menjawab pertanyaan yang jarang dijawab dari lapangan: apa yang sebenarnya terjadi pada ingatan agen ketika sistemnya hidup berbulan-bulan.

1.2 Masalah

Setiap sesi baru, agen mulai dari nol. Apa pun yang terlihat seperti ingatan sebenarnya adalah berkas yang dibaca dan dimasukkan ke context di awal sesi. Ruang context itu terbatas, dan menurut penelitian yang ada, mutu keluaran model sudah menurun jauh sebelum ruang itu penuh [rujukan 4].

Kalau semua pengetahuan ditumpuk jadi satu berkas besar, yang penting ikut tenggelam di antara yang jarang dipakai. Jawabannya, yang sudah dipakai banyak orang: pisahkan yang harus diingat setiap saat dari yang cukup dibaca saat dibutuhkan.

1.3 Dua lingkungan, satu operator

Ruang kerja agen. Enam agen kerja terspesialisasi (QA, kode, DevOps, pencatatan, riset, pengajaran) di infrastruktur pribadi, berjalan di atas OpenClaw. Tiap agen punya ruang kerja permanen berisi berkas memori. Di tulisan ini agen-agen itu disebut dengan nama lapangannya: “kuli” [buruh].

Chat engine produksi. Sistem percakapan untuk usaha kecil: pengguna mengirim pesan, sistem memetakan ke satu maksud, menjalankan, menjawab. Dipakai pengguna sungguhan sejak Juli 2026.

Lambung jangka panjang. Sebuah repositori catatan (Knowledge-OS) berisi keputusan arsitektur, jebakan yang pernah terjadi, dan prinsip operasional. Lahir 29 Mei 2026 (commit 022c5da), dibaca manusia dan mesin dengan format yang sama.

Ketiganya dijalankan satu orang. Tidak ada tim, tidak ada kelompok kontrol.

1.4 Posisi terhadap yang sudah ada

Pola “tipis di depan, lambung di belakang” bukan kontribusi tulisan ini.

[FAKTA]

Pola itu adalah konvensi bawaan OpenClaw (berkas terkurasi plus catatan harian) dan Claude Code (indeks memori plus berkas topik bertipe). Anthropic menuliskannya sebagai rekomendasi pada September 2025 dan memperkuatnya Juli 2026 dengan memangkas lebih dari 80% prompt sistem Claude Code [rujukan 5, 6]. Secara akademik, gagasan memori bertingkat untuk model bahasa sudah ada sejak MemGPT, Oktober 2023 [rujukan 1]. Standar AGENTS.md untuk instruksi agen di repositori dibuka Agustus 2025 dan dipakai lebih dari 60 ribu proyek [rujukan 7].

[FAKTA]

Sebagian besar keputusan yang dicatat di tulisan ini diambil sebelum atau bersamaan dengan tulisan-tulisan itu, tanpa membacanya. Tanggalnya ada di git (Lampiran C). Ini konvergensi independen, bukan klaim mendahului.

Kontribusi tulisan ini adalah catatan tentang apa yang terjadi ketika pola itu dijalankan: angka ongkos dari produksi, kegagalan yang bertanggal, dan dua percobaan otomasi yang ditolak beserta artefaknya. Rinciannya di Bagian 3, kaitannya ke literatur di Bagian 4.5.

2. METODE

2.1 Studi lapangan, bukan eksperimen

Ini pengamatan langsung di sistem yang berjalan sungguhan, oleh operatornya sendiri, Mei sampai September 2026. Tidak ada tolok ukur baku, tidak ada perbandingan terkontrol.

2.2 Cara bahan dikumpulkan: tagihan artefak

Bahan tulisan ini tidak dikumpulkan dari ingatan operator. Cara kerjanya:

1. Ko-dokumentator menulis “tagihan”: satu pertanyaan tertulis yang meminta artefak, bukan cerita. Tiap tagihan menyebut apa yang dicari, di mana, dan bentuk setoran yang diterima (path, timestamp, potongan log atau diff).
2. Operator meneruskan tagihan ke agen kode, satu-satunya agen yang punya akses baca ke semua ruang kerja.
3. Agen kode menyetor berkas jawaban. Yang tidak ditemukan wajib ditulis “tidak ditemukan” beserta daftar tempat yang sudah dicek.
4. Ko-dokumentator memverifikasi setoran terhadap sumber lain (repositori yang disinkronkan ke project knowledge, tangkapan layar operator) dan melabeli tiap klaim.

Sembilan tagihan dikirim antara 15 September 2026 pukul 08:31 dan 11:20 WIB. Semua berkas setoran disimpan operator.

[OBSERVASI]

Cara ini mengoreksi ingatan operator di empat tempat: insiden “history 300 ribu token” ternyata gabungan tiga kejadian berbeda (Bagian 3.6); baris-baris besar di panel pemakaian yang diyakini milik chat engine ternyata milik agen (Bagian 3.9); keterangan lisan tentang silsilah beberapa aturan meleset dari commit-nya; dan klaim kepemilikan sebuah kunci Redis ditolak oleh agen yang disebut pemiliknya, dengan berkas (Bagian 3.5). Keempatnya dicatat apa adanya.

2.3 Basis data

Jenis	Isi	Catatan
Repositori ruang kerja agen	Berkas memori semua agen, git	Privat; hanya kutipan teredaksi
Repositori chat engine	Kode, tiket, laporan QA, git	Privat; hash commit dikutip
Knowledge-OS	17 dokumen keputusan (ADR), 36 jebakan tercatat, prinsip operasional	Privat; kutipan dipilih
Panel pemakaian penyedia model	Input dan output token per panggilan, Juli sampai Agustus 2026	Kolom biaya diredaksi operator
Arsip percakapan agen	Transcript OpenClaw (teks) dan claude-cli (teks plus tool call)	Sebagian terhapus 10 Sep 2026, lihat Bagian 3.8
Catatan harian agen	Satu berkas per tanggal per agen	45 berkas dibuang 24 Jul, 29 masih ada di Trash
Artefak dreaming	76 entri malam plus 12 backfill, Juni sampai Juli	Dipulihkan dari Trash 3 Sep
Dokumen arsitektur memori	198 baris, Knowledge-OS, commit f4c5598 22 Agu 2026	Ditulis SETELAH semua insiden

2.4 Label kelas bukti

Setiap klaim membawa label di baris tersendiri, mengikuti protokol yang dijelaskan di tulisan pertama seri ini.

Ringkasnya: [FAKTA] bisa ditunjuk sumbernya; [OBSERVASI] pola yang terlihat, belum ada sebab; [HIPOTESIS | confidence: x] dugaan dengan angka keyakinan penulis; [TIDAK TAHU] tidak ada bukti dan tidak diisi tebakan. Angka keyakinan hanya di hipotesis, tidak pernah di fakta.

2.5 Redaksi

Nilai kredensial, alamat koneksi basis data, dan identitas orang selain operator dan nama agen tidak dikutip. Hash commit, nomor tiket, dan nama berkas dibiarkan karena itu penanda artefak.

2.6 Bahasa

Seluruh interaksi memakai bahasa Indonesia informal bercampur istilah lokal. Istilah teknis Inggris dibiarkan Inggris. Istilah baru diberi kurung kotak di kemunculan pertama.

3. HASIL

3.1 Pola yang dijalankan: tiga lapis, satu bus

Struktur bawaan tool: berkas terkurasi (MEMORY.md) dan catatan harian (memory/YYYY-MM-DD.md). Di atas itu operator menambahkan tiga hal.

Lapis referensi bersama. Direktori workspace-shared/ berisi playbook yang dibaca semua agen saat dipicu, bukan setiap sesi. Berkas terkurasi per agen dipangkas jadi indeks: pada 26 Juni 2026, MEMORY.md agen kode dari 125 baris jadi 88, “strictly index/pointer-only” (commit 852f8d7).

Lapis 0: bus kerja. Papan bersama antar agen di Redis (kunci ctx:* untuk berbagi konteks, task:* untuk serah terima tugas), berumur tujuh hari, sengaja tidak dianggap ingatan. Yang perlu diingat lintas hari dipetik manusia dari bus ke catatan harian.

Kurasi manual. Tidak ada promosi otomatis dari catatan harian ke berkas terkurasi. Alasannya di Bagian 3.7.

[FAKTA]

Dokumen yang merumuskan pola ini secara umum (Lampiran A memuat glosariumnya) baru ditulis 22 Agustus 2026, satu commit, tidak pernah direvisi. Semua insiden di bawah terjadi sebelum dokumen itu ada. Dokumen itu rangkuman, bukan rancangan.

3.2 Pointer adalah instruksi, bukan tautan

Aturan yang lahir dari pemakaian: cara menulis pointer menentukan dipatuhi atau tidak.

Ditulis begini	Terbaca sebagai	Hasilnya
“Detail ada di berkas X”	informasi tambahan	sering dilewat
“Baca berkas X sebelum Y. Jangan mengarang alur sendiri.”	syarat sebelum bertindak	dijalankan

Tiga bagian pointer yang berfungsi: kata kerja perintah, pemicu konkret, penutup celah.

[FAKTA]

Contoh penempatan bertanggal, 14 Juli 2026 pukul 10:14 sampai 10:23 WIB, arsip percakapan agen kode. Aturan baru soal Redis ditulis agen ke workspace/memory/. Operator bertanya: “lu simpen di mana”, “itu file dipake sapa aja”, “di warzone lu baca itu juga gak”. Jawaban agen: berkas di situ hanya dibaca satu agen, dan di kanal kerja bersama MEMORY.md tidak dimuat otomatis. Operator: “pindahin lah, itu emang seharusnya global.” Aturan pindah ke workspace-shared/ dan diberi pointer satu baris di tiap MEMORY.md agen.

[OBSERVASI]

Tiga pertanyaan operator itu adalah dua pertanyaan penyaring di Bagian 3.3, dalam bentuk lapangan: dibutuhkan siapa, kapan, dan apakah pemicunya sampai.

3.3 Aturan penempatan dan prinsip frekuensi akses

Dua pertanyaan penyaring untuk tiap informasi baru: (1) dibutuhkan setiap sesi, atau hanya saat mengerjakan hal tertentu? (2) aturan yang mengatur perilaku, atau catatan sesuatu yang pernah terjadi?

[FAKTA]

Prinsip yang dirumuskan 24 Juli 2026, dikutip dari catatan harian agen QA malam itu: “frekuensi akses yang nentuin format penyimpanan, bukan pentingnya. Sering+penting = rule terkompresi. Jarang+penting = playbook utuh + gate wajib-baca, zero compression.” Catatan yang sama menyebut prinsip itu dirumuskan bersama satu instance Claude di sesi percakapan terpisah, dan arsip sesi itu memuat percakapannya.

Kesalahan yang paling sering: catatan kejadian ditulis ke berkas prinsip. Lama-lama berkas prinsip jadi arsip.

3.4 Memori harus tinggal di ruang kerja sendiri

[FAKTA]

Sampai 26 Juni 2026 malam, memori agen hidup di direktori auto-memory Claude Code. Pukul 22:23 WIB, runtime jatuh otomatis dari satu model ke model penyedia lain (pesan sistem: “Model Fallback”). Agen di runtime baru tidak bisa membaca memori itu: menarik asumsi salah soal build, ditegur, disuruh membuka memori, menemukan jejaknya di direktori vendor lama, memindahkan 19 berkas pukul 22:37. Dua commit migrasi pukul 23:34 dan 23:46 (42433af, 852f8d7). Aturan “vendor-neutral memory” lahir 27 Juni 05:53 (2219245), dipropagasi ke semua agen pukul 06:00, semua agen mengonfirmasi lewat bus sebelum 17:16.

[FAKTA]

Kalimat operator pagi itu, verbatim: “Semalem gw ke fallback ke GPT auto amnesia auto ngawur.” Aturan kanoniknya belum pernah diubah sejak lahir.

[FAKTA]

Migrasi sebenarnya sudah dimulai 25 Juni (disebut di pesan commit). Insiden 26 Juni bukan yang melahirkan idenya, tapi yang memaksa diselesaikan malam itu dan dikunci sebagai aturan keras.

[OBSERVASI]

Yang memicu kehilangan ingatan bukan keputusan pindah tool, tapi pergantian runtime yang diam-diam. Dua hal diam bertemu: substitusi model tanpa pemberitahuan, dan memori yang terkunci di satu vendor. Portabilitas memori jadi obat untuk gejala yang sumbernya bukan di memori.

3.5 Bus kerja: dua insiden**Split-brain, 14 Juli 2026.****[FAKTA]**

Pukul 10:11 WIB operator menemukan kunci `ctx:build-chat_engine` di Redis mesin staging, bukan di Redis mesin agen. Agen kode menghapusnya 10:14 dan menulis aturan “`ctx` dan `task` hanya Redis mesin agen” (di-commit 15 Juli, `fa568e8`), lalu 10:41 menghapus semua aturan lamanya yang menyebut mesin staging (commit `91fe71c`). Percakapan penempatan aturannya ada di Bagian 3.2.

[FAKTA]

Mekanismenya bisa ditunjuk, tiga lapis. Pertama, `playbook builder` (sejak 30 Juni) dan berkas memori agen DevOps menulis perintah “append `ctx log` ke Redis `db=7`” tanpa menyebut mesin, padahal berkas yang sama mendukung dua target deploy. Kedua, agen kode memegang dua tool dengan nama ujung sama: `redis__rpush` (mesin agen) dan `mac-redis__rpush` (mesin staging), keduanya ada di context-nya sejak 2 Juli. Ketiga, kode server tool (`redis-extended.mjs`) mengambil DB default dari alamat koneksi (`staging = db 0`), tapi deskripsi parameter `db` di semua tool di-hardcode “default: 7”, termasuk instance staging. Agen yang membaca deskripsi tool mendapat dua tool yang sama-sama mengaku default 7.

[FAKTA]

Agen kode juga punya aturan sendiri (lahir antara 5 dan 10 Juli): “setelah deploy chat engine ke mac-machine, langsung append ke `ctx:build-chat_engine` (Redis `db=7`) tanpa tanya.” Nomor DB disebut, mesin tidak. Aturan itu ikut dihapus 10:41 dengan judul commit “hapus semua mac reference.”

[HIPOTESIS | confidence: 0.7]

Yang menulis ke Redis staging adalah agen kode, lewat `mac-redis__rpush` dengan `db=7`, saat salah satu redeploy 13 Juli. Ia satu-satunya agen dengan aturan auto-append, deploy 13 sampai 14 Juli terekam atas namanya, dan ia yang menghapus kunci tanpa bertanya siapa penulisnya. Panggilan tool-nya tidak selamat (Bagian 3.8), jadi ini tetap hipotesis.

[TIDAK TAHU]

Penulis pasti. Kunci di Redis staging sudah kosong saat diperiksa 15 September; isi dan penulis tiap entri tidak pernah ditampilkan sebelum dihapus.

[OBSERVASI]

Deskripsi tool adalah pointer juga, dan pointer ini menunjuk ke tempat yang salah. Agen tidak bisa tahu dari dalam bahwa “default: 7” adalah teks yang disalin, bukan nilai yang dibaca. Obat 14 Juli bukan memperbaiki teksnya, tapi melarang tool-nya untuk keperluan ini.

Diet MCP, 8 September 2026.**[FAKTA]**

Pukul 18:30 operator melapor RAM mesin agen 70% terus. Diagnosis agen DevOps 18:31: tiap giliran agen memunculkan tujuh proses penghubung basis data (MCP), dan proses itu tidak mati ketika gilirannya mati. Puncak 97 proses, sekitar 5,1 GB. Pembersihan manual 18:32; bocor kembali dalam 22 menit. Data 30 hari yang dibedah agen DevOps: hanya satu dari enam penghubung yang pernah dipakai (agen kode 103 kali, agen QA 2 kali), lima lainnya nol. Keputusan 19:30: keenam penghubung dimatikan, akses basis data pindah ke skrip yang dijalankan saat dibutuhkan. Proses per giliran turun dari tujuh jadi satu. Notifikasi ke semua agen lewat bus (`ctx:mcp-diet-db-tools`), lima agen mengonfirmasi dalam 30 menit.

[TIDAK TAHU]

Perintah yang dipakai untuk menghitung 97 dan 103. Arsipnya terhapus dua hari kemudian (Bagian 3.8).

[OBSERVASI]

Bus-nya benar sejak awal. Yang bocor adalah cara menyambung ke bus: penghubung yang selalu dimuat untuk semua agen, termasuk yang tidak pernah memakainya. Pola yang sama dengan lapisan tetap yang terlalu gemuk, di lapisan infrastruktur.

3.6 Matryoshka: ringkasan yang memakan dirinya sendiri

[FAKTA]

Chat engine punya tabel `ai_memory_summary` sejak hari pertama, 24 Mei 2026 (commit 8634dbb, fb09101). Fungsinya menyimpan ringkasan sesi. Penulisnya, `build_memory_summary`, bukan peringkas: tiap giliran ia menempelkan seluruh ringkasan lama di bawah baris “Previous summary:” tanpa pemangkasan. Kodenya identik dari lahir sampai dihapus, 51 hari.

[FAKTA]

Sampel dari basis data uji tanggal 26 Mei, hari kedua, sudah bersarang dua lapis, dengan isi giliran pertama muncul dua kali. Pada 14 Juli, agen QA mencatat di tiket v2.BUG-5: satu sesi uji berisi ringkasan 628.916 karakter (sekitar 150 ribu token), 411 lapis “Previous summary:”, satu petunjuk kaleng terkubur 84 kali. Efek: pengklasifikasi maksud macet di satu maksud, timeout 30 detik, evaluasi tiga model hari itu dinyatakan tidak sah.

[TIDAK TAHU]

Prosedur pengukuran angka itu. Tidak tersimpan dan tidak diingat penulisnya. Pada 15 September dua mesin diperiksa baca-saja (riwayat shell, log, arsip, catatan harian): nihil. Bentuk yang paling mungkin diturunkan dari kode, panjang kolom dan jumlah substring, dicapai terpisah oleh dua agen [HIPOTESIS | confidence: 0.8]. Angka dipakai “sebagaimana dicatat QA 14 Juli 2026”, dengan catatan harian QA tanggal yang sama sebagai sumber kedua.

[FAKTA]

Ada dua jalur pembaca. Jalur pertama diberi gerbang di perakitan prompt dan praktis tidak pernah terbuka. Jalur kedua, di lima lokasi worker, menyuntik ringkasan ke prompt interpretasi setiap pesan tanpa gerbang. Audit pagi 14 Juli (tiket v2.TBD-1, 09:13) hanya melihat jalur pertama dan menyimpulkan “bom laten”. Tiket sore (18:03) menemukan jalur kedua: bomnya aktif. Catatan harian QA hari itu: “pembaca kedua yang gw MISS di audit pagi, retracted.”

[FAKTA]

Optimasi 30 Juni (tiket RT-4: batas history 12 jadi 3 pesan, ringkasan dilewati bila ada pesan terbaru) hanya menyentuh jalur pertama. Jalur kedua hidup 14 hari lagi. Pencabutan total 14 sampai 15 Juli, dua ronde (88133b5, 8a3dd10), migrasi 20260714_0009 menghapus tabelnya.

[OBSERVASI]

Insiden ini tidak masuk daftar jebakan di Knowledge-OS. Ia ada di arsip tiket, laporan mingguan, dan salinan untuk atasan, tapi tidak naik ke lumbung. Kurasi manual yang tulisan ini gambarkan bolong di tempat yang paling penting.

3.7 Dreaming: otomasi kurasi yang ditolak

[FAKTA]

Fitur “dreaming” OpenClaw diaktifkan di akar ruang kerja, jadi mengenai semua agen. Tiap malam pukul 03:00 WIB ia menulis entri diari dari catatan harian, dan satu putaran “Deep Sleep” memeringkat lalu mempromosikan kandidat ke berkas terkurasi tanpa kurasi manusia. Artefak yang tersimpan: 76 entri malam (16 Juni sampai 14 Juli 2026), 12 entri backfill (3 sampai 18 Juni), satu Deep Sleep yang mempromosikan 6 kandidat.

[FAKTA]

Dua pengamatan langsung dari artefak, diverifikasi dua pihak pada 3 September: refleksi yang dihasilkan bersifat templat, satu kalimat identik berulang di 7 dari 12 hari backfill, dan beberapa kandidat ditandai sendiri sebagai tidak jelas (baris artefak 107, 108, 176); register diari kembali ke suara bawaan liris berbahasa Inggris, tidak seperti register kerja agen.

[FAKTA]

Fitur dimatikan 24 Juli 2026, artefaknya dibuang dari lima ruang kerja pukul 18:57 sampai 19:06 (sekitar 8,8 MB), lalu dipulihkan operator dari Trash 3 September untuk tulisan ini. Pada 8 Agustus, satu entri hasil promosi otomatis dihapus dari berkas terkurasi agen QA dengan alasan yang dicatat: “recalls=0, gak pernah kepakai, sumber daily note-nya udah ke-trash.”

[OBSERVASI]

Alasan penolakan yang dicatat operator: agen yang memorinya dikurasi otomatis terdeteksi drift kembali ke perilaku bawaan pabrik. Kesimpulan operasionalnya: kurasi memori adalah fungsi penilaian, bukan pemipaan.

3.8 Lapisan mentah tidak bertahan

Tiga kali, dengan tiga sebab berbeda.

[FAKTA]

24 Juli 2026, dibuang manusia. Audit memori lintas agen, ketok operator per item. 45 catatan harian dari tiga agen dibuang (agen QA: 29 berkas, 3 Juni sampai 23 Juli, commit 878fd58; agen kode: 15; agen DevOps: 1), total 1.811 baris dihapus, 42 baris naik ke berkas terkurasi dan bersama. Alasan yang dicatat: aturan berharganya sudah dipromosikan semua. Pemicu hari itu: penggantian model embedding pukul 17:30 membuat folder dreaming mencemari hasil pencarian memori.

[FAKTA]

10 September 2026, dihapus agen. Agen DevOps menghapus 2.860 berkas arsip percakapan claude-cli (225 MB) dengan `find -delete`, bukan Trash, tanpa persetujuan eksplisit. Arsip claude-cli adalah satu-satunya tempat panggilan tool dan perintah shell agen tercatat; arsip OpenClaw hanya menyimpan teks. Aturan “jangan hapus tanpa ketok eksplisit” lahir sesudahnya.

[FAKTA]

Berkelanjutan, rotasi container. Log worker chat engine berganti tiap build. Log tanggal 14 Juli sudah tidak ada pada 15 September; log yang tersisa mulai 14 September.

[FAKTA]

Bukti balik. Pada 15 September, agen QA ditanya tentang catatan harian 13 sampai 16 Juli dan menjawab “filenya gak pernah ada.” Berkasnya ada di Trash, dibuang agen QA sendiri 24 Juli. Dua di antaranya (14 dan 15 Juli) dipulihkan sebagai sumber kedua untuk Bagian 3.6 dan 3.9. Pada 22 Juli, dua hari sebelum membuang, agen QA sendiri mencatat risikonya: “record bolong bikin gw salah klaim ‘ga pernah ada’.”

[OBSERVASI]

Yang bertahan hanya yang sempat naik jadi tiket atau aturan. Itu argumen paling kuat untuk distilasi, dan datang dari kegagalan sendiri. Tapi bukti balik di atas menunjukkan yang dibuang kadang justru yang dibutuhkan. Ketegangan ini dibahas di Bagian 4.4.

3.9 Ongkos: dua sistem, satu panel

Chat engine dan agen riset memakai gateway penyedia model yang sama pada Juli 2026, jadi keduanya muncul di satu panel pemakaian. Dua tangkapan layar operator (kolom biaya diredaksi) dan satu record runtime agen memberi angka dari satu sumber.

[FAKTA]

Chat engine, tipis di depan sejak 30 Juni (commit e52e489: klasifikasi maksud hanya membaca dua berkas). Panel 20 Juli: 3.081 token input. Panel 12 Agustus, 19 panggilan: 3.274 sampai 4.592, datar sepanjang jam. Log aplikasi 1 September: 2.383 tanpa history, 2.494 dengan 3 pesan history; history menambah sekitar 110 token. Minggu 36: 2.130 sampai 2.803.

[FAKTA]

Runtime agen, tidak tipis. Panel 22 Juli pukul 17:19 sampai 17:23: sebelas panggilan agen riset naik dari 42.468 ke 55.961, sekitar 1,3 ribu per giliran. Record runtime agen pada 17:19 mencocokkan baris pertama persis (input 42.468, output 76). Diagnosis agen sendiri 25 Juli: prompt sistem sekitar 49 ribu karakter plus skema tool 32 ribu karakter disuntik ulang tiap giliran. Panel 29 Juli: sekitar 100 ribu per giliran. Dua kali overflow context tercatat (2 dan 31 Juli).

[FAKTA]

Teguran operator 29 Juli pukul 19:41, verbatim: “Edun lu 1 turn ampe 100k token, mau rampog lu.” Itu akta lahir doktrin “anti-rampok-token” yang kemudian dipegang semua agen.

[OBSERVASI]

Operator sama, periode sama, panel sama. Yang beda hanya cara merakit prompt. Sistem yang menerapkan tipis-di-depan memakai sekitar 3 ribu dan datar; sistem yang tidak, 42 sampai 116 ribu dan menumpuk. Selisih sekitar 12 kali lipat, dengan catatan modelnya berbeda sehingga hitungan token tidak persis sebanding; bentuknya (datar vs menumpuk) tidak bergantung model.

[HIPOTESIS | confidence: 0.7]

Sebelum 14 Juli, chat engine sendiri pernah termasuk yang gemuk: ringkasan 150 ribu token disuntik tiap panggilan (Bagian 3.6). Catatan harian QA 14 Juli menulis “sebagian call gede kemungkinan = call interpretation chat engine bawa matryoshka summary.” Tidak ada tangkapan layar panel sebelum 14 Juli untuk memastikannya.

[FAKTA]

Meterannya sendiri pernah mati. 2 September 2026: model produksi mengirim usage kosong, semua kolom token null di log, karena pustaka klien hanya meminta usage bila alamat gateway tidak diset. Gateway lain kebetulan selalu mengirim usage dan menutupi bug ini. Diperbaiki commit 8bdf2e5. Angka dari log aplikasi sebelum tanggal itu untuk model produksi adalah null, bukan nol.

3.10 Konvergensi ke produksi: chat engine

Pola yang sama dijalankan di chat engine tanpa diubah bentuknya. Dokumentasinya disusun terpisah oleh agen kode.

Bentuk. Berkas konteks sebagai teks biasa di dalam kode. Lapisan yang selalu ada: identitas, karakter, persona, memori, dan satu indeks kemampuan. Kontrak tiap kemampuan di berkas sendiri, dibaca saat maksudnya terpilih. Kasus yang mudah tertukar dipisah ke berkas pembeda. Tahap pemilihan maksud hanya membaca dua berkas.

Gerbang deterministik didahulukan. Pesan berpenanda terstruktur, konfirmasi draft, pertanyaan tertunda: diputuskan tanpa model. Model hanya dipanggil bila semua gerbang tidak mengambil alih.

Ingatan routing dengan sakelar yang sengaja dimatikan. Sistem mencatat tiap keputusan routing dan bisa memutuskan ulang pesan serupa tanpa model, di balik dua ambang. Pada saat dokumen ini ditulis, pemakaiannya dimatikan: korpus belum cukup tebal untuk menopang skor keyakinan. Pencatatan berjalan sejak awal, pemakaian ditahan. “Kumpulkan dulu, pakai belakangan.”

Pola ditegakkan, bukan disepakati. Kesesuaian indeks, kontrak, dan kode diperiksa saat proses mulai berjalan. Dokumen usang menjatuhkan proses, bukan menyesatkan pembaca.

[OBSERVASI]

Jalur akses memori di chat engine juga pernah bercabang: jalur dispatcher membawa 12 pesan history (default), jalur graph membawa 3 (batas RT-4). Batas yang dipasang di satu jalur tidak sampai ke jalur lain. Nilai batasnya juga berbeda per lingkungan (produksi 5, staging 10).

3.11 Lumbung: Knowledge-OS

[FAKTA]

Lahir 29 Mei 2026 pukul 08:19 WIB, dua commit dalam sebelas detik: README dan AGENTS.md. Prinsip operasionalnya ada sejak detik pertama: “capture first, organize later”, “decision beats discussion”, “AI should read from durable notes, not memory vibes”, “don’t over-engineer the knowledge base to the point of avoiding it”. AGENTS.md: “Decision logs override brainstorming notes.”

[FAKTA]

Per 15 September: 17 dokumen keputusan (ADR), tidak satu pun digantikan sejak Mei. Tiap ADR punya seksi Forward Implication yang memuat kalimat penolakan eksplisit, contoh ADR-001: “Any time AI suggests writing to product tables, that suggestion is wrong. Push back.” Jebakan tercatat 36 entri, format tetap: halusinasi yang pernah terjadi, realitanya, cara memverifikasi. Dua aturan penutupnya: “Read the actual file before claiming what’s inside. Code rots; memory rots faster.” dan “never paraphrase a preference you cannot quote.”

[FAKTA]

ADR-012, 29 Mei 2026: kalimat “Data tersimpan.” hanya boleh keluar bila draft benar-benar tersimpan. Aturan “jangan klaim sebelum terjadi” ada di lapisan antarmuka pengguna sejak minggu pertama.

[OBSERVASI]

Dalam satu minggu yang sama, 24 sampai 30 Mei, operator memasang bom ringkasan di chat engine (Bagian 3.6) dan memasang lumbung disiplin di Knowledge-OS. Lumbung itu bukan reaksi ke insiden; ia lahir lebih dulu sebagai kebiasaan, dan insidennya tetap terjadi. Disiplin di lumbung tidak otomatis menular ke kode.

4. DISKUSI

4.1 Ingatan adalah yang dibaca ulang, bukan yang disimpan

Ukuran penyimpanan tidak pernah jadi masalah di dua lingkungan ini. Yang jadi masalah selalu ukuran yang dibaca tiap panggilan: lapisan tetap agen (49 ribu karakter prompt sistem), ringkasan yang disuntik tiap pesan (150 ribu token), tujuh penghubung yang dimuat tiap giliran. Tiga kali, obatnya sama: memindahkan dari “dibaca selalu” ke “dibaca saat dipicu”.

4.2 Kurasi adalah penilaian, bukan pemipaan

Dua percobaan otomasi ditolak dari dua arah. Dreaming mengotomasi promosi ke atas dan menghasilkan refleksi templat plus drift register. Matryoshka mengotomasi akumulasi tanpa batas dan meledak. Yang tersisa di tengah adalah manusia yang membaca catatan harian, memutuskan per item, dan mencatat alasannya. Itu mahal, dan pada 24 Juli ternyata juga salah di satu tempat (Bagian 3.8).

4.3 Jalur akses memori bercabang diam-diam

Empat insiden, satu pola: dispatcher 12 vs graph 3; pembaca ber-gerbang vs tanpa gerbang; Redis mesin agen vs mesin staging; dan pada 6 Juli, dua daemon Docker di mesin staging, sehingga agen kode men-deploy ke daemon A sementara port layanan masih dipegang stack lama di daemon B, dan seluruh pengujian agen QA hari itu mengenai stack yang salah. Di keempatnya, ada satu jalur yang diketahui dan dibatasi, dan satu jalur lain yang tidak diketahui dan tidak dibatasi. Perbaikan di jalur pertama tidak sampai ke jalur kedua, dan jalur kedua yang menyebabkan kerusakan.

[HIPOTESIS | confidence: 0.6]

Ini bukan kebetulan tiga kali. Setiap kali “memori” dibaca dari lebih dari satu tempat di kode, batas yang dipasang di satu tempat akan tertinggal. Yang membantu bukan batas yang lebih ketat, tapi satu titik baca.

4.4 Distilasi versus verbatim: ketegangan yang belum selesai

Penelitian terkontrol Januari 2026 menunjukkan potongan verbatim mengalahkan artefak hasil ekstraksi untuk percakapan panjang [rujukan 9]. Praktik di tulisan ini justru membuang verbatim dan menyimpan ekstraksi, dengan alasan yang masuk akal (Bagian 3.8), dan tujuh minggu kemudian membutuhkan verbatim yang dibuang.

Kedua sisi punya bukti. Distilasi membuat agen bisa kerja dengan context tipis. Verbatim membuat klaim bisa diperiksa belakangan. Tulisan ini tidak punya jawabannya; yang bisa disumbangkan adalah satu kasus di mana keduanya terjadi pada sistem yang sama, bertanggal.

4.5 Konvergensi independen, bertanggal

Semua keputusan di Bagian 3 punya commit atau catatan harian bertanggal. Dibandingkan dengan literatur (Lampiran C): keputusan tipis-di-depan (30 Juni), vendor-neutral memory (27 Juni), dan prinsip frekuensi akses (24 Juli) jatuh sebelum atau tepat bersamaan dengan tulisan Anthropic “new rules” (24 Juli) dan gelombang tool memori berbasis markdown (Agustus sampai September 2026). Keputusan menghapus entri berdasar `recalls=0` (8 Agustus) muncul empat bulan setelah paper “When to Forget” (April) [rujukan 8] tanpa membacanya. Knowledge-OS (29 Mei) lahir sebulan setelah pola LLM Wiki Karpathy (April) [rujukan 10].

Ini bukan klaim mendahului. Ini bukti bahwa pola yang sama ditemukan ulang oleh praktisi tanpa akses ke literatur, dan bahwa jalur penemuan ulangnya (lewat kegagalan) menghasilkan detail yang literatur tidak punya: umur bom, jalur ganda, register drift, dan lapisan mentah yang hilang.

4.6 Keterbatasan

Satu operator, tidak ada kontrol. Angka token dari dua sumber (panel penyedia dan log aplikasi) dengan model yang berbeda; bentuk kurva sebanding, angkanya tidak persis. Prosedur pengukuran matryoshka tidak tersimpan. Arsip yang terhapus 10 September membuat beberapa “tidak ditemukan” di tulisan ini bersifat permanen. Ko-dokumentator adalah instance model bahasa yang sama jenisnya dengan yang diamati; labelnya di kepala dokumen menyatakan itu. Dan ingatan operator terbukti keliru tiga kali selama pengumpulan bahan; yang tidak tertangkap artefak mungkin masih keliru.

5. KESIMPULAN

Pola tipis-di-depan sudah ada di mana-mana. Yang belum ada adalah catatan tentang apa yang terjadi ketika pola itu dijalankan berbulan-bulan tanpa tim: ringkasan yang memakan dirinya sendiri selama 51 hari, otomasi kurasi yang menghasilkan templat, jalur akses yang bercabang tiga kali, dan lapisan mentah yang hilang tiga kali. Tulisan ini menyerahkan catatan itu apa adanya, dengan tanggal dan label kelas bukti, termasuk bagian di mana ingatan operatornya salah.

LAMPIRAN A. Glosarium

Istilah	Artinya
Context	Ruang teks yang dibaca model saat bekerja. Terbatas, seperti meja kerja.
Context awal	Kumpulan berkas yang otomatis dibaca setiap sesi dimulai.
Externalize	Memindahkan isi yang jarang dipakai keluar dari context awal, ke berkas terpisah.
Pointer	Satu baris di context awal yang menunjuk ke berkas terpisah, lengkap dengan kapan berkas itu harus dibuka.
Distilasi	Menyaring catatan harian yang menumpuk, mengambil yang layak disimpan jangka panjang.
Vendor lock	Data tersandera satu tool, sulit pindah.
Bus kerja	Papan bersama antar agen untuk serah terima; berumur pendek, bukan ingatan.
Matryoshka	Ringkasan yang menyisipkan ringkasan sebelumnya utuh, bersarang tanpa batas.
Dreaming	Fitur konsolidasi memori otomatis malam hari di OpenClaw.
Kuli	Agen kerja terspesialisasi di ruang kerja agen.

LAMPIRAN B. Perbandingan ruang kerja agen dan chat engine

Peran	Ruang kerja agen	Chat engine
Aturan perilaku yang selalu berlaku	berkas perilaku di akar ruang kerja	identitas, karakter, persona
Indeks penunjuk	daftar pointer satu baris per rujukan	indeks kemampuan
Prosedur rinci, dibaca saat dipicu	berkas di direktori referensi	kontrak per maksud
Pembeda kasus yang mudah tertukar	catatan jebakan pada berkas terkait	berkas pembeda per maksud
Pemicu pembacaan	kata kerja perintah pada pointer	maksud terpilih hasil klasifikasi

LAMPIRAN C. Garis waktu

Tanggal	Kejadian di lapangan	Artefak	Literatur sejamin
24 Mei 2026	Tabel ringkasan lahir (bom dipasang)	8634dbb, fb09101	MMPO 28 Mei
29 Mei	Knowledge-OS lahir; ADR-012	022c5da	Karpathy LLM Wiki, Apr
26-27 Jun	Fallback runtime, amnesia, migrasi, aturan vendor-neutral	42433af, 852f8d7, 2219245	
29-30 Jun	Chat engine tipis di depan; RT-4	e52e489, a038531	
6 Jul	Dua daemon Docker di staging; pengujian mengenai stack lama	arsip WarZone	
14 Jul	Split-brain Redis; RCA matryoshka; aturan Redis	V2.BUG-5, fa568e8, 91fe71c	
14-15 Jul	Ringkasan dicabut total	88133b5, 8a3dd10	
22 Jul	Panel: agen 42-56 ribu per giliran	record runtime	
24 Jul	Audit memori, dreaming dibasmi, prinsip frekuensi akses	878fd58 + daily note	Anthropic “new rules”, 24 Jul
29 Jul	Teguran 100 ribu token, doktrin anti-rampok	arsip percakapan	
8 Agu	Entri auto-promote dihapus, recalls=0	daily note QA	“When to Forget”, Apr
12 Agu	Panel: chat engine 3,3-4,6 ribu, datar	tangkapan layar	
22 Agu	Dokumen arsitektur memori ditulis	f4c5598	
2 Sep	Meteran token diperbaiki	8bdf2e5	
3 Sep	Artefak dreaming dipulihkan, diverifikasi dua pihak		
8 Sep	Diet MCP	backup config, ctx:mcp-diet-db-tools	OKF Agent Memory, Sep
10 Sep	2.860 arsip terhapus	daily note DevOps	
15 Sep	Sembilan tagihan artefak	berkas setoran	

LAMPIRAN D. Rujukan

[1] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

[2] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023.

[3] Lin, K., Snell, C., Wang, Y., Packer, C., Wooders, S., Stoica, I., & Gonzalez, J. E. (2025). Sleep-time Compute: Beyond Inference Scaling at Test-time. arXiv:2504.13171.

[4] Hong, K., Troynikov, A., & Huber, J. (2025). Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma technical report, July 2025.

[5] Anthropic (2025). Effective context engineering for AI agents. Engineering blog, September 2025.

[6] Shihpar, T., Anthropic (2026). The new rules of context engineering for Claude 5 generation models. 24 July 2026.

[7] AGENTS.md specification (2025), Agentic AI Foundation, Linux Foundation; adoption figures per arXiv:2604.21090.

[8] Simsek, B. (2026). When to Forget: A Memory Governance Primitive. arXiv:2604.12007.

[9] Verbatim Chunks Beat Extracted Artifacts: A Controlled Ablation of Memory Representations for Long LLM Conversations (2026). arXiv:2601.00821. Penulis belum diverifikasi.

[10] Karpathy, A. (2026). LLM Wiki (gist), April 2026.

[11] Meta-Cognitive Memory Policy Optimization for Long-Horizon LLM Agents (2026). arXiv:2605.30159. Penulis belum diverifikasi.

Catatan: rujukan [9] dan [11] dibaca dari abstrak; nama penulis akan dilengkapi sebelum versi 1.0.

Authority by upbringing, not by degree.